Project: Cracking MD5 hashes

Phillip Pinter

May 16, 2024

1 Introduction

The goal of this project is to crack MD5 hashes using a brute force attack on the GPU. Hashing refers to the process of generating a fixed-sized output from an input of variable size using the mathematical formulas known as hash functions [1]. It is important that the output is a unique hash for every input but this isn't always the case. This is called hash collision and this is an important metric to evaluate an algorithm.
Hashing is used in a variety of situations but in this project the focus is on the hashing of password strings.

It is important that hashing passwords is done with a one-way hashing algorithm. This is a hashing algorithm where it is impossible to revert the mathematical formula. So you can calculate the hash of a password string but it is impossible to calculate the password string of a hash.

This makes it very useful for storing passwords. If a hacker found its way into a database which stores passwords as regular strings, they could easily know which user used which password and could use that information to access other accounts of that user since there are a lot of people that use the same password for multiple accounts or use easy to crack variations. If they for example use "Netflix123!" on Netflix, it is likely they use "Amazon123!" on Amazon.

Hashing makes it a lot more difficult to find the password string of a user in a data-breach but this project will show that it isn't impossible even though one-way hashing algorithms are impossible to revert.
So the project will use brute force hash cracking to find the original string of a given MD5 hash.

2 What is brute force hash cracking?

Brute force in this context means trying every single possibility until you find the solution. When you have a hash you want to know the password string of, it is impossible to revert that hash to its original string input but it is not impossible to hash every possible input to see which input hashed to the same one you're trying to crack. This is called brute force hash cracking.

In brute force hash cracking you want to calculate as many hashes a second as possible which makes it a great project to showcase the strength of parallelizing with the GPU. But to focus on the parallelization, the project will calculate the hashes of all possible character combinations of a certain maximum length. So this way the performance is easier to measure since you work with a fixed instruction.

3 What is the MD5 algorithm?

The MD5 (Message-Digest Algorithm 5) is a widely used cryptographic hash function that produces a 128-bit (32 hexadecimal characters) hash value from an input message of variable size. The input message can contain any printable ASCII character, even a single space will have its own unique hash. So the resulting hash will always have the same 16 byte size. This means there are $2^{128} = 3.4 \times 10^{38}$ possible hashes. The input string "KU Leuven" for example will return the hash "14157161e255788e45fb422178c6db82".

MD5 has been historically used for storing hashed passwords but due to certain vulnerabilities it is not recommended anymore for that purpose. The main reason is that it is calculated relatively quickly which makes it more vulnerable to brute force attacks.

4 The implementation in CUDA

4.1 How is an MD5 hash calculated?

The calculation of a single MD5 hash is a sequential task since each step requires the result of the previous step so this part cannot be parallelized. It is still extremely important that this implementation is optimized since the brute force attack needs to calculate millions if not more hashes a second so a small computation difference in the algorithm could result in a huge difference in the brute force attack.

In MD5 you start with your input string. It can contain any printable ASCII character, for example "`@a7%$`". After this you convert the input in it's binary representation according to the ASCII conversion chart where each character is 8 bits long. The character 'A' for example is represented by "01000001".
The characters of a char* string in C++ are already stored in their binary form so there is no need to convert the string which will save time compared to the first implantation where the binary values were stored as strings themselves.

Decimal	Binary	Octal	Hex	ASCII
64	01000000	100	40	@
65	01000001	101	41	A
66	01000010	102	42	B
67	01000011	103	43	C
68	01000100	104	44	D
69	01000101	105	45	E
70	01000110	106	46	F
71	01000111	107	47	G

Figure 1: ASCII table. Source: [4]

After this, a 512 bit message needs to be created. The last 64 bits are reserved to represent the length of the message (so there are 448 bit remaining). The message starts with the bits of the input string, then follows a single '1' bit and then all zeros to fill the rest of the 448 bits. For the last 64 bits you add the binary value of the length of the input in bits (and all zeros for the remaining bits). So if the input has a length of 22 characters, it will have $22 \times 8 = 176$ bits and the binary representation of 176 (0b10110000) is used. After these steps the 512 bit message is complete but it has to be split in 16 32-bit words. These words are called the M-values and every operation in the algorithm, 1 of these values is used.

A very important note when doing this which isn't often clarified online, is that the MD5 algorithm uses the little-endian notation where the least significant byte (so 8 bits) are on the left instead of on the right. This is done for every 32-bit word. So "01010100 01101000 01100101 01111001" will need to change to "01111001 01100101 01101000 01010100". So it is important you shift the bits accordingly to get little-endian.
This 512-bit message is created in the "prepare_start" device function. The focus here was to use as much bit-wise operations as possible to improve performance like shifting 5 times left instead of dividing by 32.

After this, 2 constant arrays are declared called K and S. They have both 64 elements where K's elements are 32-bit binary values and S's elements are integers. K is computed at the beginning of the program with the function:

$$K[i-1] = \lfloor 2^{32} \cdot |\sin(i+1)| \rfloor$$

With i starting with 1 and ending with 64. S are the rotation constants. In each operation in the algorithm, a result is shifted to the left by a certain amount. The elements in S dictate how far is shifted to the left and these are defined in the official documentation of MD5 [2].
Since both K and S are constants (so they are only read) and every single thread needs these values multiple times, they are stored in constant memory on the GPU for faster access.

The algorithm itself is made up of 4 rounds and each round has 16 operations so there are a total of 64 operations. Each round has its own distinct function, F, G, H and I respectively:

$$F(X,Y,Z) = (X \wedge Y) \vee (\neg X \wedge Z) \tag{1}$$
$$G(X,Y,Z) = (X \wedge Z) \vee (Y \wedge \neg Z) \tag{2}$$
$$H(X,Y,Z) = X \oplus Y \oplus Z \tag{3}$$
$$I(X,Y,Z) = Y \oplus (X \vee \neg Z) \tag{4}$$

where:

- $\wedge$ denotes the bitwise AND operation.
- $\vee$ denotes the bitwise OR operation.
- $\oplus$ denotes the bitwise XOR operation.
- $\neg$ denotes the bitwise NOT operation.

The algorithm starts with 4 initialization vectors and they are defined as follows (in their hexadecimal form for readability). Note that they are written in the little-endian notation as well:

$$A = 0x67452301 \tag{5}$$
$$B = 0xEFCDAB89 \tag{6}$$
$$C = 0x98BADCFE \tag{7}$$
$$D = 0x10325476 \tag{8}$$

The algorithm can now start. In the function "CalculateHashResult" you can see the different operations. In every operation, all of the values of a, b, c and d are read and one of them is manipulated. This means that every operation is depended on the previous operation which makes this part of the code not suitable to parallelize.
Each operation uses one of these functions: FF, GG, HH and II. These functions all do the same thing with the only difference, the function it calls inside. FF calls the round function F, GG calls G, HH calls H and II calls I. The formula of each function is as follows:

$$x \mathrel{+}= X(y,z,k) + M[i] + K[i] \tag{9}$$
$$x = \text{rotateLeft}(x, S[i]) \tag{10}$$
$$x \mathrel{+}= y \tag{11}$$

where:

- x, y, z and k are the first, second, third and fourth argument of the function respectively.
- X is the corresponding round function
- M[i] is the the M word at index i in the array from the 512-bit message
- K[i] is the constant at index i in the array with the constants coming form the sine function.
- S[i] is the constant at index i in the array with the rotation constants. The rotateLeft function performs a circular left shift by S[i] positions.

The 4 first arguments in the function change every operation from position but you can see that every 4 operations, the order in which they are first called returns. The fifth argument is the index of the operation which is used to get the K and S value. The last argument is the M word. Since the M array only has 16 words (512 bits with 32 bits per word) the index is reset to zero every round.

When all the rounds are completed, the last step is performed. Here the results of the words a, b, c and d are added with their corresponding starting value. Since all the calculations were done in the little-endian notation, the result need to be converted back to the big-endian notation (byte0 switches with byte3 and byte1 switches with byte2.

The hash has now finished calculating. The hash is just the combination by appending the words a, b, c and d. So for the input string "KU Leuven", the words will be respectively "14157161", "e255788e", "45fb4221" and "78c6db82" which will result in the complete hash of "14157161e255788e45fb422178c6db82".

4.2 The brute force attack in CUDA

As discussed before, this project will calculate the hash of every possible character combination with a given maximum length (so the smaller lengths as well). If the maximum length is set to 4, the combination 'aaaa' is tried, as well as 'a'. The metrics that are measured are the average computing time to calculate all the hashes, the total amount of combinations that were computed and the average amount of hashes that are calculated each second.

Firstly, the theadID of the current thread is calculated which is followed by the calculation of the stride. This is the total amount of threads in the grid and determines the interval between combinations a thread is iterating through when there are more combinations than there are threads.
After this, the length of the combination the thread is going to compute is calculated. Then the thread is given a local ID which represents the index within its current length range. Finally the input string for that thread is generated character by character. The character is selected from the alphabet which contains all the printable ASCII characters. Since this alphabet is used by every thread to calculate its input string and the values are read-only, constant memory is used for to store the alphabet.

When the input string is generated, the MD5 hashing can begin. First the 512-bit message block is computed by calling the prepare_start function which will store the result in 'messageBlock'. Since the thread calculates its hash by itself, local memory is used. After this the hash itself is calculated by calling CalculateHashResult and the result will be stored in the words A, B, C and D which can be compared to the hash you are trying to find (the hash can be separated in 4 words as well). But as mentioned before, the focus of this project was on computing all possible combinations.
The hash has now finished computing and to verify the combinations are computed correctly, the test array is used to print the hash of a given string to verify that combination has been used. The last combination with a maximum length of 4 is "~~~~" and the resulting hash for that combination is "ed790e13 4796eb70 4dd092dd e146a792" which is the correct hash. For the performance testing this is commented out.

4.3 The main function

The main function starts with initializing the K constants which are computed with the sine formula. After this the K and S values, as well as the alphabet are copied into constant memory as discussed before.
Then the amount of total combinations are calculated with the formula:

$$\text{Total Combinations} = \sum_{i=1}^{N} A^i$$

where A is the size of the alphabet and N the maximum length for the combinations.

After this you can specify the amount of threads per block and the amount of blocks to use. These can be changed to find which settings work best with the program. Then the timer to time the kernel and its corresponding variables are added. The kernel is executed multiple times to get a more accurate reading but be moderate with how much since the amount of combinations change exponentially with the maximum length (with a maximum length of 6 for example, there are over 740 billion combinations). Each iteration the kernel is called with the specified attributes and it is timed.

5 Testing performance

As mentioned before, the amount of combinations grow exponentially with the maximum length so for testing only the maximum lengths 1 through 5 are tested, where for the averages lengths 1 through 4 are executed 10 times and length 5, 3 times. The tesing was done on a Lenovo Legion Slim5 14 laptop with an RTX4060 but note that during the testing, the laptop was connected to it's 140W USB-C port instead of it's 170W power brick which reduced the performance of the GPU to about 82% of its maximum performance.
As a reference, the testing is done firstly with 1 thread doing everything. maxLen=5 is not testing due to the time it would take to compute (multiple hours):

```
---- maxLen = 1 -----
Total combinations: 95
Average time to complete kernel: 7.584768 ms
Average hashes per second: 12 525 hashes/s
---- maxLen = 2 -----
Total combinations: 9 120
Average time to complete kernel: 154.356735 ms
Average hashes per second: 59 083 hashes/s
---- maxLen = 3 -----
Total combinations: 866 495
Average time to complete kernel: 10805.397461 ms (= 10.81 s)
Average hashes per second: 80 190 hashes/s
---- maxLen = 4 -----
Total combinations: 82 317 120
Average time to complete kernel: 1095570.375000 ms (= 18.3 min)
Average hashes per second: 75 136 hashes/s
```

After trying different threads per block and amount of blocks, the following settings gave the best performance: 1024 threads per block and 255 blocks. These settings gave the following results:

```
---- maxLen = 1 -----
Total combinations: 95
Average time to complete kernel: 0.744960 ms
Average hashes per second: 473 572 hashes/s
---- maxLen = 2 -----
Total combinations: 9 120
Average time to complete kernel: 0.907568 ms
Average hashes per second: 23 827 007 hashes/s
---- maxLen = 3 -----
Total combinations: 866 495
Average time to complete kernel: 11.658150 ms
Average hashes per second: 75 940 280 hashes/s
---- maxLen = 4 -----
Total combinations: 82 317 120
Average time to complete kernel: 1003.235962 ms (= 1 s)
Average hashes per second: 82 076 404 hashes/s
---- maxLen = 5 -----
Total combinations: 7 820 126 495
Average time to complete kernel: 100617.562500 ms (= 1.68 min)
Average hashes per second: 77 723 200 hashes/s
```

The maximum lengths of 1 and 2 have a lower amount of hashes per second but this is expected since in these scenarios there will be more threads than combinations so not all the threads are used. Because of this, the focus will be on the lengths 3, 4 and 5.

When all the threads are used, the amount of hashes per second stay fairly consistent. This is mainly because the task of a thread stays the same. The padding (creating the 512-bit message block) isn't impacted by using larger strings and the block has a constant size of 512 bits so the threads will use it's values the same way.

These results do show that parallelizing the brute force attack improves performance significantly. For a maximum length of 3, it was about 950 times faster than using a single thread and for the maximum length of 4, it was about 1100 times faster.

Comparing with tools like Hashcat [3], it could be improved by a factor of 100 so there are a few improvements that could be made.

The threads are already running asynchronously (since they don't depend on each other) and they only read from constant (or local) memory so using coalesced memory access is not applicable here. The biggest bottleneck is most likely the way threads calculate their input string first before computing the hash. Some possible improvements that could be made are found in the next section.

6 Shortcomings and possible improvements

With more time, the following functionality could be added to the program:

- Be able to input a hash and get the cracked input string back.
- Add the possibility to brute force using a list of commonly used passwords instead of trying every single combination.
- Implement multiple hashing algorithms and let the user choose which algorithm his hash needs to be cracked with.
- Be able to set constrains on what combinations are generated. If you for example already know it ends with "123!", it will only try the combinations that end with that.

Additionally, the following possible improvements could be made:

- The biggest bottleneck is most likely the fact that each threads needs to calculate its input string before calculating its hash. Making this more optimal or calculating it beforehand would be very helpful.
- Dynamically adjust the workload distribution among threads and blocks based on their progress and computational load.
- Offloading certain tasks to the CPU, such as managing combination generation for large datasets or pre-processing input data, can free up GPU resources for te more computationally intensive tasks.

References

[1] GeeksforGeeks. What is hashing. Available at: https://www.geeksforgeeks.org/what-is-hashing/. Accessed on: May 15, 2024.

[2] Ronald L. Rivest. The md5 message-digest algorithm. Available at: https://www.ietf.org/rfc/rfc1321.txt. Accessed on: May 15, 2024.

[3] Open Source. Advanced password recovery. Available at: https://hashcat.net/hashcat/. Accessed on: May 16, 2024.

[4] Donald Weiman. Ascii conversion chart. Available at: https://web.alfredstate.edu/faculty/weimandn/miscellaneous/ascii/ascii_index.html. Accessed on: May 15, 2024.